

NexusPay

Cross-Platform Banking and FinTech Transaction System

Project status: Student project / functional prototype

Architecture: React single-page application + Express REST API + MySQL and MongoDB

Primary purpose: Demonstrate a secure, full-stack digital banking experience, including authentication, account management, internal fund transfers, beneficiaries, notifications, audit activity, and transaction search.

1. Overview

NexusPay (also referred to in the UI as **PulseBank**) is a cross-platform banking and FinTech application. It is designed as a realistic banking prototype rather than a production bank: users can register, authenticate, view currency accounts and balances, manage beneficiaries, transfer funds between internal accounts, review transaction history, receive notifications, and search financial activity.

The project uses **polyglot persistence**:

- **MySQL 8** is the system of record for relational and financial data: users, accounts, transfers, and beneficiaries.
- **MongoDB 7** stores flexible document-oriented data: notifications, activity/audit logs, and device/session records.

The backend is currently a modular monolith. Its modules have clear boundaries so that selected components can be extracted into services in a future phase without prematurely introducing microservice complexity.

Main goals

- Create a responsive, modern banking interface for desktop and mobile.
- Protect financial operations with validation, database transactions, precision-safe amounts, row locks, and optimistic locking.
- Maintain activity and notification records for user visibility and auditing.
- Demonstrate professional engineering practices: TypeScript, input validation, API documentation, Docker, tests, and database migrations.

Scope statement

NexusPay models internal transfers in a learning environment. It does **not** claim to integrate with a regulated bank, card network, payment rail, or KYC provider. It must not be used to handle real funds or real customer data without substantial security, compliance, operational, and legal work.

2. Features

Feature area	Current capability
Authentication	Registration, login, JWT issuance, password-reset flow, password hashing
Accounts	Automatic default USD account, multiple currency accounts, account balance and account-number display
Transfers	Internal account-to-account transfer, balance validation, self-transfer prevention, reference IDs, transaction history
Beneficiaries	Create, view, update, delete, and favourite saved payees; bank-code fields for SWIFT/IFSC/routing values
Notifications	Transaction, security, system, and promotional notifications; read/unread filtering and bulk mark-as-read
Audit activity	Login, registration, account, beneficiary, transfer, and security event logging
Search	TF-IDF and cosine-similarity search over transactions and activity records
Dashboard	Account overview, balance display controls, money-in/money-out summary, cash-flow visualization, recent activity, quick actions
User experience	Responsive navigation, dark/light theme support, form validation, toasts, loading states, and error boundaries
Developer experience	Swagger/OpenAPI API docs, Docker Compose, TypeScript, migrations, Vitest, property-based test tooling

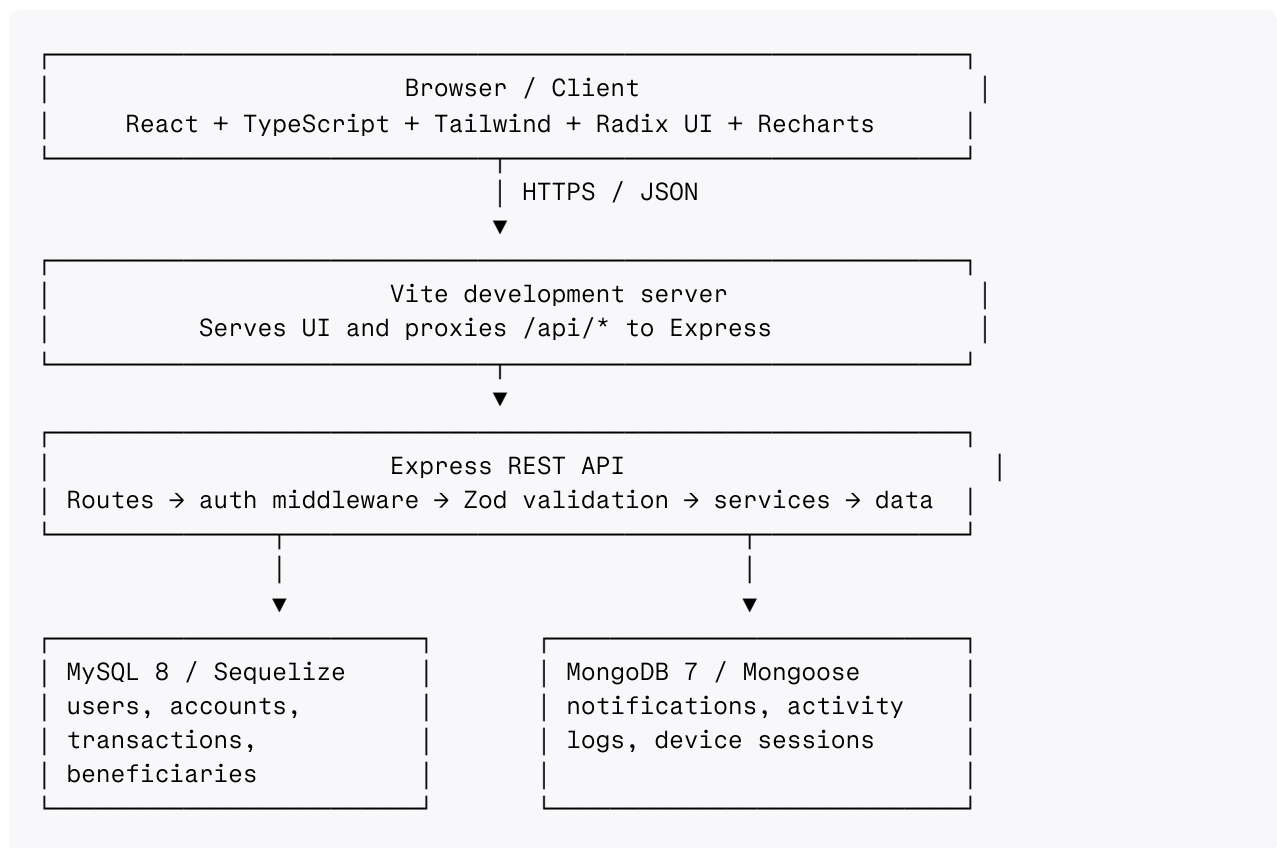
3. Technology stack

Layer	Technology	Responsibility
Frontend	React 19, TypeScript	Component-based web interface
Build tool	Vite	Development server, API proxy, optimized frontend build
Styling	Tailwind CSS, Radix UI	Responsive styling and accessible UI primitives
Forms	React Hook Form, Zod	Form state and client-side validation
Routing	Wouter	Lightweight single-page routing
Visualization	Recharts	Dashboard cash-flow and financial charts
Backend	Node.js 20+, Express.js, TypeScript	REST API, domain services, validation, authentication
Relational database	MySQL 8, Sequelize, mysql2	Financial and relational system-of-record data
Document database	MongoDB 7, Mongoose	Logs, notifications, sessions, flexible metadata

Layer	Technology	Responsibility
Authentication	JWT, bcrypt	Credential verification and authenticated requests
Precision	decimal.js	Decimal-safe application arithmetic
API documentation	Swagger/OpenAPI 3	Discoverable and testable REST API contract
Testing	Vitest, fast-check	Unit and property-based test foundations
Deployment	Docker, Docker Compose	Reproducible local and production-like environment

4. Architecture

Logical design



Runtime flow

1. The user interacts with a React page or reusable UI component.
2. The frontend calls a REST endpoint under `/api` using JSON.
3. Vite proxies development requests to the Express server. In production, Express serves the built frontend and API from one deployment unit.
4. Express authenticates the request where necessary, validates input with Zod, and invokes the relevant service.
5. Services enforce business rules and persist data in MySQL or MongoDB.

6. The API returns a structured success or error response to the React client.

Project structure

```
cross-platform-banking-ui/
├── client/                                # React/Vite frontend
│   ├── src/components/                  # Reusable UI and banking components
│   ├── src/contexts/                   # Authentication and theme state
│   ├── src/hooks/                      # Custom hooks
│   ├── src/pages/                      # Login, signup, home and recovery pages
│   └── src/lib/                        # Client utilities
├── server/                             # Express backend
│   ├── config/                        # Environment and Swagger configuration
│   ├── db/                           # Connections, ORM/ODM models, SQL migrations
│   ├── middleware/                    # JWT and rate-limiting middleware
│   ├── routes/                       # HTTP endpoint definitions
│   ├── services/                     # Domain and business logic
│   ├── validation/                   # Zod request schemas
│   └── index.ts                      # Server entry point
├── shared/                            # Shared types and utilities
├── tests/                             # Automated tests
├── Dockerfile
├── docker-compose.yml
└── vite.config.ts
```

5. Frontend

Pages

Page	Purpose
Home	Dashboard with account overview, cash flow, activity, quick actions, and notifications
Login	Username/password authentication with remember-me behavior
Sign up	Account registration with validation
Forgot password	Starts the password-reset workflow
Reset password	Accepts a reset token and new password
Not found	Fallback route for invalid URLs

UI design

The user interface uses reusable Radix-based components and Tailwind CSS. It supports responsive layouts, a mobile navigation drawer, dark/light theme selection, form feedback, toast messages, balance masking, account-number copying, and transfer dialogs.

The frontend has an `AuthContext` that owns login, registration, logout, and session rehydration. Protected routes prevent unauthenticated users from accessing banking views.

Development proxy

During development, the frontend runs on port 3000; API requests starting with `/api` are proxied to the Express server on port 3001.

```
server: {
  port: 3000,
  proxy: {
    "/api": {
      target: "http://localhost:3001",
      changeOrigin: true,
    },
  },
}
```

This avoids hard-coding a backend origin in browser requests and keeps the frontend API interface stable across environments.

6. Backend modules

Module	Main responsibility	Typical operations
Authentication	Identity and credential lifecycle	Register, login, profile lookup, reset-token creation, password reset
Accounts	Account lifecycle and balance queries	Create account, list accounts, retrieve details and balance
Transactions	Monetary transfer processing and history	Transfer, validation, locking, transaction creation, pagination/filtering
Beneficiaries	Saved payee ownership and management	Add, list, update, delete, favourite
Notifications	User alert lifecycle	List, filter, mark read, mark all read, delete
Search	Cross-record retrieval	Similarity ranking across activity and transaction text

API conventions

- Base path: `/api`
- Payload format: JSON
- Protected endpoints require `Authorization: Bearer <access-token>`.
- Input is validated before service execution.
- Pagination and filters are used for collection endpoints where applicable.
- Swagger UI is available at `/api/docs` when the server is running.
- Health check: `GET /api/health`.

Key endpoints

Domain	Endpoints
Auth	POST /auth/register, POST /auth/login, POST /auth/forgot-password, POST /auth/reset-password, GET /auth/me
Accounts	POST /accounts, GET /accounts, GET /accounts/:id, GET /accounts/:id/balance
Transactions	POST /transactions/transfer, GET /transactions
Beneficiaries	POST /beneficiaries, GET /beneficiaries, GET /beneficiaries/:id, PUT /beneficiaries/:id, DELETE /beneficiaries/:id
Notifications	GET /notifications, POST /notifications/:id/read, POST /notifications/read-all, DELETE /notifications/:id
Search	GET /search?q=<query>&type=transactions activity all

7. Data model

MySQL: financial system of record

Entity	Important fields	Role
Users	UUID, username, email, password hash, KYC status, reset token expiry	Customer identity and credentials
Accounts	UUID, user ID, account number, decimal balance, ISO currency, version ID	Customer balances and account ownership
Transactions	UUID, sender/receiver account IDs, amount, currency, type, status, reference, timestamp	Transfer record and history
Beneficiaries	UUID, user ID, payee name, account details, bank details, favourite flag	Saved transfer destinations

Important data rules:

- A user can have only one account for each currency: `UNIQUE (user_id, currency)`.
- Account numbers and user emails are unique.
- Monetary values use `DECIMAL (15,4)`; floating-point numbers are not used for balances or transfer amounts.
- IDs are UUIDs stored compactly as `BINARY (16)` in MySQL and converted for API use.
- Transaction records are intended to be append-only; status handling remains under service-layer control.

MongoDB: flexible event and communication data

Collection	Role	Important indexes
Notifications	User-facing transaction, security, system, and promotional messages	User/read/date; user/type/date; TTL expiry
Activity logs	Audit and activity events with flexible metadata	User/date; user/category/date; text search fields
Device sessions	Refresh-session and device metadata	User/device/session-expiry lookup as required

MongoDB document metadata permits different event types to store different details without changing a fixed relational schema.

8. Transfer design

Objective

A transfer must not lose money, duplicate money, or leave one account updated while the other remains unchanged. The project achieves this using a MySQL database transaction, row locks, validation, and an optimistic-lock version field.

Transfer workflow

1. Validate request fields and authenticate the sender.
2. Find sender and receiver accounts.
3. Reject self-transfer and invalid account/currency conditions.
4. Begin a database transaction.
5. Lock sender and receiver account rows using `SELECT ... FOR UPDATE`.
6. Re-check balance after locks are acquired.
7. Create a pending transaction record.
8. Debit sender with `version_id` verification.
9. Credit receiver and increment its `version_id`.
10. Mark the transaction completed.
11. Commit the database transaction.
12. Create activity-log and notification documents after the successful commit.

Concurrency controls

- **Row-level locks:** prevent simultaneous updates to the same account rows during a transfer.
- **Optimistic locking:** `version_id` is checked during account updates. A mismatch indicates that another operation changed the account first.
- **Rollback:** if any transfer step fails before commit, MySQL rolls back all financial updates.
- **Decimal arithmetic:** prevents binary floating-point rounding errors.

Limitation to document

Financial data is committed in MySQL before post-transfer activity and notification writes are attempted in MongoDB. If the application fails after the MySQL commit, a notification or log may be missed even though the financial transfer succeeded. A production-grade evolution should implement a relational **outbox pattern** and background worker to retry these post-commit events reliably.

9. Security design

Current controls

Control	Implementation
Password protection	bcrypt hashing with 12 rounds; plaintext passwords are not stored
Authentication	JWT signed with HS256 and configured expiry
Password reset	Random 32-byte raw token sent to the user; only a SHA-256 hash is stored, with a one-hour expiry
Request validation	Zod schemas on client forms and server endpoints
Authorization	JWT middleware attaches the authenticated user; ownership checks protect user-specific resources
Rate limiting	Auth: 5 requests/15 min; transfers: 10/15 min; general API: 100/15 min
Transfer integrity	Database transactions, locks, optimistic version control, balance checks, self-transfer prevention
Data precision	DECIMAL database fields and decimal.js rather than JavaScript floating-point arithmetic
Production transport	HTTPS is required in production

Security improvements before real deployment

- Move to short-lived access tokens plus rotating refresh tokens in HttpOnly, Secure, SameSite cookies.
- Avoid storing long-lived bearer tokens in localStorage because an XSS vulnerability can expose them.
- Add multi-factor authentication or step-up confirmation for sensitive transfers.
- Add an idempotency key to transfer requests to prevent duplicate transfers on retries or double-clicks.
- Add Helmet, strict CORS configuration, CSP, CSRF handling when cookies are used, security-event alerting, and secret rotation.
- Use a managed secret store; never commit database credentials or JWT keys.
- Introduce security scanning, dependency updates, centralized redacted logs, backups, monitoring, and incident procedures.

10. Search

The search service uses tokenization, stop-word removal, term frequency, inverse document frequency, and cosine similarity to rank activity and transaction text against a query.

Example: a search for `payment sent` scores records that are semantically similar to those terms rather than performing only an exact phrase match. The service applies a similarity threshold—default 0.3—and returns top-ranked results.

This is a useful demonstration algorithm. At larger scale, consider database-native full-text search or a dedicated search engine; this would reduce the need to fetch and score large numbers of documents in application memory.

11. Local setup

Prerequisites

- Node.js 20 or later
- npm or the package manager selected by the repository
- Docker and Docker Compose, or separately installed MySQL 8 and MongoDB 7
- Environment variables for database connections, JWT secret, and mail/reset configuration

Typical workflow

```
# Install dependencies
npm install

# Start MySQL and MongoDB if Docker Compose is configured
npm run docker:up

# Run development servers
npm run dev
```

The exact script names must be verified in `package.json`. The frontend normally serves on `http://localhost:3000`, while the Express API normally serves on `http://localhost:3001` in development.

Required environment variables

Names may vary by implementation. A typical local `.env` file should contain values equivalent to:

```
NODE_ENV=development
PORT=3001
JWT_SECRET=replace-with-a-long-random-development-secret
JWT_EXPIRES_IN=7d
MYSQL_HOST=localhost
MYSQL_PORT=3307
```

```
MYSQL_DATABASE=nexuspay
MYSQL_USER=nexuspay_user
MYSQL_PASSWORD=local-password
MONGODB_URI=mongodb://localhost:27017/nexuspay
CLIENT_ORIGIN=http://localhost:3000
```

Do not commit `.env` files with real credentials. Commit a sanitized `.env.example` file instead.

Production build

```
npm run build
```

The production process builds the Vite frontend and configures Express to serve the generated static application while also exposing `/api/*` endpoints.

12. Testing strategy

The project includes Vitest and fast-check. The test suite should expand around the banking invariants rather than only UI snapshots.

Unit tests

- Zod validation rules and amount parsing.
- Password and reset-token services.
- Account ownership checks.
- Beneficiary CRUD authorization.
- Transaction status-transition rules.
- Search tokenization and ranking behavior.

Integration tests

- Successful transfer changes sender and receiver balances correctly.
- Insufficient balance rolls back all updates.
- Self-transfer is rejected.
- Cross-user access to account, beneficiary, or notification data is denied.
- Concurrent transfer attempts maintain a correct final balance.
- A duplicate idempotency key returns the original result rather than creating a second transfer.

Financial invariants

For every completed internal transfer, verify:

```
sender balance decrease + receiver balance increase = 0
```

For accounts without overdraft support, verify:

```
balance >= 0 after every successful or failed operation
```

13. Roadmap

Priority 1 — Complete and stabilize

- Finish every UI-to-API flow and align response types with frontend expectations.
- Add consistent loading, empty, retry, and error states.
- Complete Swagger schemas and examples.
- Add integration tests for all protected endpoints.
- Verify actual session/token persistence and Authorization-header behavior.

Priority 2 — Financial correctness

- Add request idempotency for transfers.
- Add an immutable double-entry ledger (`ledger_entries`) rather than relying only on mutable account balances.
- Add a reliable outbox event table and worker for MongoDB notifications/activity logs.
- Define formal state transitions and model reversals as compensating transactions.
- Add reconciliation checks and concurrency test coverage.

Priority 3 — Database evolution

PostgreSQL is a sensible future alternative to MySQL, particularly if the project wants native UUIDs, `TIMESTAMPZ`, `JSONB`, partial indexes, advanced full-text search, richer constraints, and powerful SQL tooling. The migration should be intentional, tested, and accompanied by data reconciliation—not simply a driver replacement.

A simplified target architecture is either:

```
PostgreSQL for financial data + MongoDB for logs/notifications + outbox worker
```

or, for operational simplicity:

Priority 4 — Security and operations

- Add refresh-token rotation and session revocation.
- Add MFA or transaction confirmation.
- Configure CSP, CORS, CSRF protections, secure cookies, and headers.
- Add CI/CD, container scanning, dependency scanning, database backups, health checks, structured logs, metrics, and tracing.

14. Portfolio summary

NexusPay demonstrates more than a visual banking dashboard. It combines a responsive React interface with a modular Express API, relational transaction processing, document-oriented logging, JWT authentication, API documentation, Docker-based deployment, and an explicit approach to money precision and transfer concurrency.

The strongest next milestone is to transform the existing transfer flow into a more auditable financial core: idempotent requests, immutable ledger entries, reliable event delivery, robust automated tests, and secure session management. That evolution would make NexusPay a particularly strong full-stack FinTech portfolio project.